

TALLER ANALÍTICO 1: OPERACIONES CON TENSORES (20 MIN)

Discutan y resuelvan:

1. Si ejecutamos la instrucción `recorte = imagen[100:200, 300:400, 1]` sobre una imagen de 1920x1080. ¿Cuáles son las dimensiones exactas (shape) de la variable `recorte` resultante y qué información específica contiene?
2. ¿Por qué es computacionalmente más eficiente aislar un canal usando *Slicing* (ej. `img[:, :, 0]`) en lugar de crear dos ciclos `for` anidados para iterar sobre la altura y anchura? (Pista: piensen en la arquitectura de la memoria).

1. Las dimensiones (shape) del recorte serían (100, 100).
100:200 en filas → toma 100 filas (de la fila 100 a la 199).
300:400 en columnas → toma 100 columnas (de la columna 300 a la 399)
El 1 al final no es un rango, es un número fijo → significa "solo quiero el canal 1" (el canal Verde)
Como se pide un solo canal específico (no un rango), Python "aplana" esa dimensión y el resultado ya no es un tensor 3D sino una matriz 2D de 100x100.
¿Qué contiene? Es un pedacito cuadrado de 100x100 píxeles, recortado de una zona específica de la foto, pero mostrando solo la información del color verde de esa zona (sin el azul ni el rojo)
2. En términos simples: usar `for` es como revisar los píxeles uno por uno, a mano, con instrucciones que Python tiene que interpretar cada vez que pasa por el ciclo. Eso es lento porque Python es un lenguaje "interpretado" y cada vuelta del ciclo tiene un costo extra de procesamiento.
En cambio, el Slicing (`img[:, :, 0]`) le dice a NumPy: "tráeme todo este bloque de una vez". NumPy está construido internamente en un lenguaje mucho más rápido (C), y además los datos de la imagen están guardados en la memoria de la computadora de forma continua y ordenada, como si fueran cajones organizados en fila. Entonces, en lugar de ir cajón por cajón preguntando "¿qué hay aquí?", NumPy agarra el bloque completo de una sola vez. Por eso es mucho más rápido, sobre todo con imágenes grandes que tienen millones de píxeles.

TALLER DE LABORATORIO 1: TRANSFORMACIÓN DE ESPACIOS (25 MIN)**Reto en Python:**

1. Abran su IDE. Creen un píxel BGR de prueba completamente amarillo intenso:
`pixel = np.array([0, 255, 255]).`
2. Calculen matemáticamente su valor en escala de grises usando la fórmula ponderada superior (escribanlo usando operaciones básicas de NumPy, no usen funciones de OpenCV todavía).
3. Impriman el resultado. ¿Qué valor de gris (intensidad de 0 a 255) arroja un amarillo puro?
4. Ahora, usen la función optimizada de OpenCV: `img_gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)` sobre una imagen real descargada de internet para corroborar el proceso.

The screenshot shows a code editor with three tabs: `indexacion.py`, `laboratorio_trasnformacion_espacios.py` (active), and `Laboratorio_Final_Kernel.py`. The active tab contains the following Python code:

```

1  import numpy as np
2
3  pixel = np.array([0, 255, 255]) # [Azul, Verde, Rojo]
4
5  print("Píxel BGR original:", pixel)
6
7  B = pixel[0]
8  G = pixel[1]
9  R = pixel[2]
10
11  gris = (0.114 * B) + (0.587 * G) + (0.299 * R)
12
13  print("Valor en escala de grises:", gris)

```

Below the code editor is a terminal window with tabs for Problems, Output, Debug Console, Terminal (active), and Ports. The terminal shows the following commands and output:

```

• @gomezzuley19-debug →/workspaces/IA-2_-Talleres (main) $ source /workspaces/IA-2_-Talle
• (.venv) @gomezzuley19-debug →/workspaces/IA-2_-Talleres (main) $ /workspaces/IA-2_-Talle
  ratorio_trasnformacion_espacios.py"
  Píxel BGR original: [  0 255 255]
  Valor en escala de grises: 225.93
○ (.venv) @gomezzuley19-debug →/workspaces/IA-2_-Talleres (main) $ █

```

3. ¿qué valor de gris arroja un amarillo puro?

El amarillo puro da un gris de 225.3, un valor bastante alto (cercano a 255, que es blanco). Esto tiene sentido porque el amarillo se forma combinando verde y rojo al máximo, y el ojo humano (y por tanto esta fórmula) es muy sensible a esos dos colores. Por eso el amarillo se percibe como un color "brillante", casi tan claro como el blanco, incluso sin tener nada de azul.

4.

The screenshot displays the Visual Studio Code interface with a Python script in the editor and its execution output in the terminal.

Explorer Panel: Shows the file structure of the project, including files like `indexacion.py`, `Laboratorio_Final_Kernel.py`, `taller 1 IA_Zuley Gomez.pdf`, and files related to `Taller 2` such as `laboratorio_trans_espacio_prueba_opencv.py`, `laboratorio_transformacion_espacios.py`, `paisaje.jpg`, `imagen_gris.jpg`, `LICENSE`, and `requirements.txt`.

Editor Panel: Displays the Python script `laboratorio_trans_espacio_prueba_opencv.py` with the following code:

```
1 import cv2
2
3 imagen = cv2.imread('Taller 2/paisaje.jpg')
4 img_gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
5
6 cv2.imwrite('imagen_gris.jpg', img_gris)
7 print("Conversión completada. Forma de la imagen en grises:", img_gris.shape)
```

Terminal Panel: Shows the execution output of the script:

```
@gomezzuley19-debug →/workspaces/IA-2-Talleres (main) $ source /workspaces/IA-2-Talleres/.venv/bin/activate
(.venv) @gomezzuley19-debug →/workspaces/IA-2-Talleres (main) $ /workspaces/IA-2-Talleres/.venv/bin/python "/workspaces/IA-2-Talleres/Taller 2/laboratorio_trans_espacio_prueba_opencv.py"
Conversión completada. Forma de la imagen en grises: (452, 678)
(.venv) @gomezzuley19-debug →/workspaces/IA-2-Talleres (main) $
```

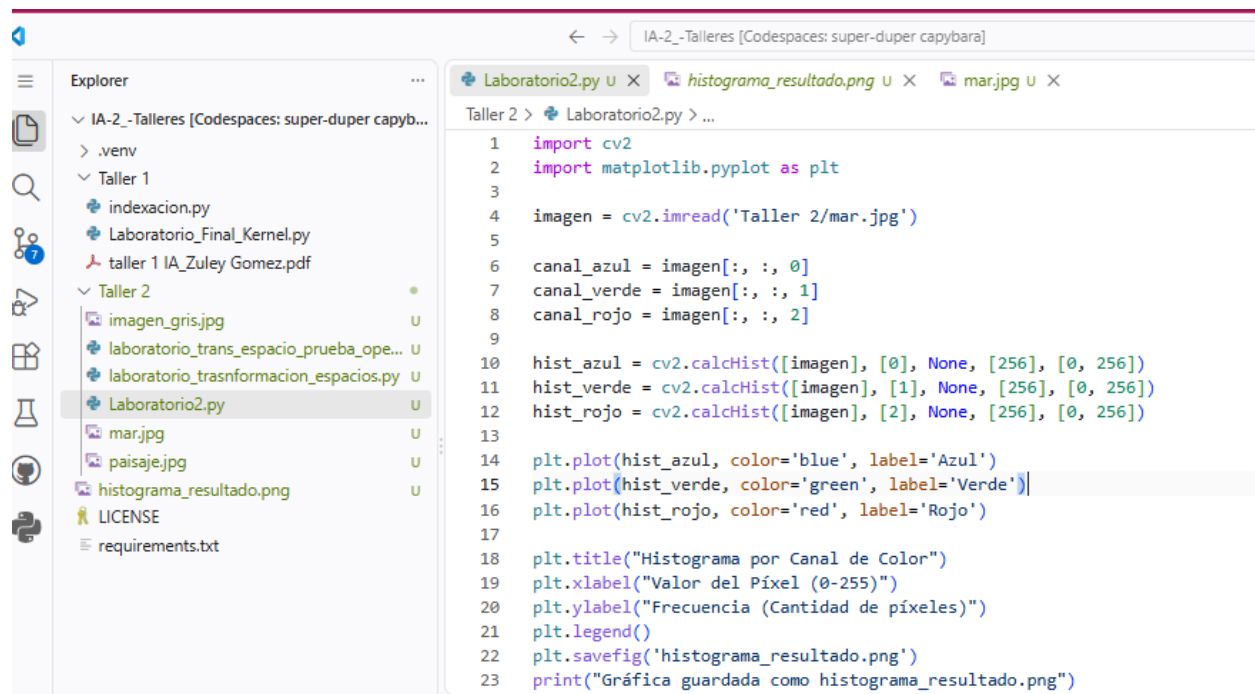
Image Preview Panel: Displays the original image `paisaje.jpg`, which is a colorful landscape with a river, trees, and flowers.



TALLER DE LABORATORIO 2: ANÁLISIS ESTADÍSTICO (30 MIN)

Misión Final:

1. Carguen una imagen RGB de su elección en Python.
2. Separen la imagen en sus 3 canales (B, G, R).
3. Calculen el histograma de **cada canal por separado**.
4. Utilicen matplotlib para graficar los tres histogramas superpuestos en la misma figura (usando colores azul, verde y rojo para las líneas respectivamente).
5. Concluyan analizando el gráfico: ¿Cuál es el color dominante en la iluminación general de su fotografía?

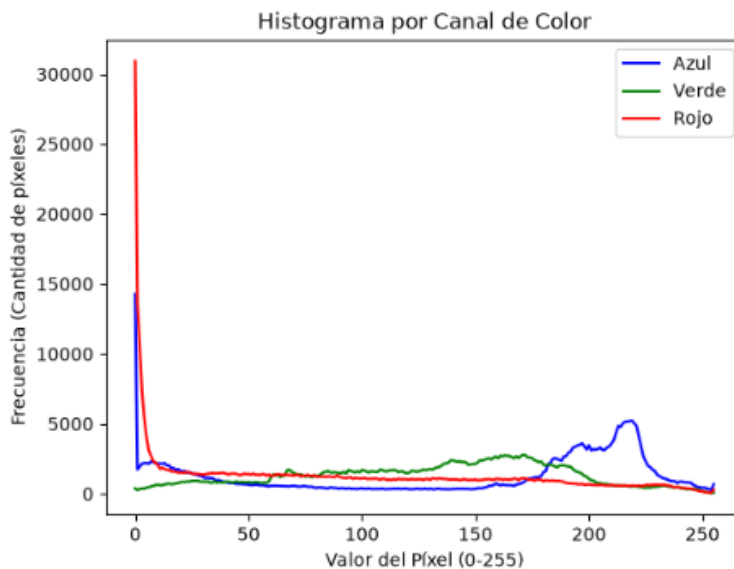


The screenshot shows a Jupyter Notebook environment with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure with files like 'indexacion.py', 'Laboratorio_Final_Kernel.py', 'taller 1 IA_Zuley Gomez.pdf', 'Taller 2', 'imagen_gris.jpg', 'laboratorio_trans_espacio_prueba_ope...', 'laboratorio_trasformacion_espacios.py', 'Laboratorio2.py', 'mar.jpg', 'paisaje.jpg', 'histograma_resultado.png', 'LICENSE', and 'requirements.txt'. The code editor shows the following Python code:

```

1  import cv2
2  import matplotlib.pyplot as plt
3
4  imagen = cv2.imread('Taller 2/mar.jpg')
5
6  canal_azul = imagen[:, :, 0]
7  canal_verde = imagen[:, :, 1]
8  canal_rojo = imagen[:, :, 2]
9
10 hist_azul = cv2.calcHist([imagen], [0], None, [256], [0, 256])
11 hist_verde = cv2.calcHist([imagen], [1], None, [256], [0, 256])
12 hist_rojo = cv2.calcHist([imagen], [2], None, [256], [0, 256])
13
14 plt.plot(hist_azul, color='blue', label='Azul')
15 plt.plot(hist_verde, color='green', label='Verde')
16 plt.plot(hist_rojo, color='red', label='Rojo')
17
18 plt.title("Histograma por Canal de Color")
19 plt.xlabel("Valor del Píxel (0-255)")
20 plt.ylabel("Frecuencia (Cantidad de píxeles)")
21 plt.legend()
22 plt.savefig('histograma_resultado.png')
23 print("Gráfica guardada como histograma_resultado.png")

```

 histograma_resultado.png

Problems Output Debug Console Terminal Ports

```
(.venv) @gomezzuley19-debug →/workspaces/IA-2 -Talleres (main) $ /workspaces/IA-2 -Talleres/.venv/bin/python "/workspaces/IA-2 -Talleres/histograma.py"
(.venv) @gomezzuley19-debug →/workspaces/IA-2 -Talleres (main) $ ||
```

Conclusion: El color que más destaca en la foto es el azul, lo cual se nota en el histograma con un pico alto entre 200 y 230. Esto pasa porque la imagen es un paisaje de costa donde el cielo y el mar turquesa ocupan casi todo el espacio. Por su parte, el pico del canal rojo cerca de cero no significa que haya rojo, sino que muestra las zonas oscuras de las rocas y las plantas donde no hay luz roja. En resumen, analizando los datos, se confirma que el azul es el color dominante de toda la escena.